Deep Learning Mathematics

A Comprehensive Guide: From Forward Propagation to Gradient Descent

---

1. Network Architecture & Topology

Neural networks are organized in layers of interconnected "neurons." For our study, we utilize a **2-3-2 Feedforward Network**:

- **Input Layer (X):** 2 features (e.g., $[x_1, x_2]$).
- **Hidden Layer:** 3 neurons using **ReLU** (Rectified Linear Unit) to solve non-linear problems.
- **Output Layer ($\hat{Y}$):** 2 neurons predicting final classes or values.

2. Mathematical Initialization

Weights (W) are the learnable parameters. We start with specific initial matrices:

$$W^1 \text{ (Layer 1): } [[0.5, 0.3], [0.2, 0.8], [0.7, 0.1]]$$
$$W^2 \text{ (Layer 2): } [[0.4, 0.6, 0.2], [0.9, 0.1, 0.5]]$$

Input Vector (a^0): [1.2, 0.8] | **Target (y):** [0.7, 0.4]

3. Forward Propagation Mechanics

Data flows forward via linear combinations followed by non-linear activations:

1. **Linear Combination:** $Z = W \cdot A_{prev} + b$
2. **Activation Function:** $A = f(Z)$ (where f is ReLU: $max(0, Z)$)

This process maps the input space into a feature space that the network can interpret to make predictions.

4. Quantifying Error: Loss Functions

We use **Mean Squared Error (MSE)** to calculate the "cost" of our current weights:

$$\text{Loss } (L) = 1/2m \, \Sigma \, (y - \hat{y})^2$$

Calculated Errors: $\mathbf{E_1 = -0.348}$, $\mathbf{E_2 = -0.904}$. Total MSE = **0.469**.

5. Backpropagation & The Chain Rule

To reduce the error, we need the gradient of the loss with respect to every weight. We use the **Chain Rule**:

$$\partial L / \partial W = (\partial L / \partial a) \times (\partial a / \partial z) \times (\partial z / \partial W)$$

This allows us to "assign blame" for the error to each weight in the network, starting from the output and moving backward.

6. Optimization via Gradient Descent

We update weights to move against the gradient (toward the minimum loss):

$$W_{new} = W_{old} - \alpha \times \nabla W$$

The Impact of Learning Rate (α)

Learning Rate	Behavior	Outcome
Low (0.01)	Small steps	Slow convergence; may get stuck.
Optimal (0.1)	Steady descent	Balanced speed and accuracy.
High (1.0)	Large jumps	Risk of overshooting the minimum.

7. Advanced Optimization Techniques

Regularization

Techniques like **L2 Regularization** (Weight Decay) add a penalty to the loss function to prevent weights from becoming too large, which helps prevent **Overfitting**.

Vanishing Gradient Problem

In deep networks, gradients can shrink to nearly zero during backpropagation. Using **ReLU** instead of Sigmoid helps because ReLU's gradient is 1 for all positive inputs, ensuring signal persists through many layers.

8. Conclusion

The mathematical foundation of Deep Learning ensures that as long as we can define a differentiable loss function, the network can iteratively improve its internal representation of the world through the cycle of forward passes and backpropagation.